



INTERNSHIP FINAL TECHNICAL REPORT

SMART SUN TRACKER

Real-Time Sun Detection and Tracking
Using Raspberry Pi and OpenCV

Students	Matin Shirani zadeh Amin Moradyan
Supervisor	Mr. Hesam Ahmadi
Organization	SAPA
Department	SAINO
Date	Summer 2026

A lightweight embedded computer-vision system for real-time detection and tracking of the sun using a Raspberry Pi Zero 2 W, a Raspberry Pi Camera, and an OpenCV-based brightness and ROI-tracking pipeline.

Abstract

This report presents the **Smart Sun Tracker**, an internship project that detects and tracks the sun in real time on a low-cost embedded platform. The system uses a **Raspberry Pi Zero 2 W** and a **Raspberry Pi Camera**, with software written in **Python**, **OpenCV**, **NumPy**, and **Picamera2**. Each frame is converted to HSV, the Value channel is thresholded with a dynamic brightness threshold, the resulting mask is cleaned with morphological operations, and contours are analysed to estimate the sun's centre and an equivalent radius. A **region-of-interest (ROI)** tracking mechanism searches only a small window around the previously detected sun position, which greatly reduces the number of pixels processed per frame. A periodic full-frame scan and a lost-target counter allow the tracker to recover from drift or temporary loss. The final system runs as a single Python program and shows a real-time visualisation of the detected sun, its centre, the tracking status, the dynamic threshold, and the active ROI. The report also discusses the limitations of the brightness-based approach, especially the possibility of false positives from bright non-sun objects.

Keywords: Sun Detection, Sun Tracking, Computer Vision, OpenCV, Raspberry Pi, Embedded Systems, Image Processing, ROI Tracking, Real-Time Processing.

Contents

Abstract	1
1 Introduction	6
1.1 Project Background	6
1.2 Problem Statement	6
1.3 Project Objectives	7
1.4 Scope of the Project	7
2 Hardware and Software Platform	8
2.1 Raspberry Pi Zero 2 W	8
2.2 Raspberry Pi Camera	8
2.3 Camera and Image Acquisition Pipeline	8
2.4 Embedded Processing Constraints	9
2.5 Software Stack	9
3 System Architecture	11
3.1 Overall Architecture	11
3.2 Image Acquisition	12
3.3 Processing Pipeline	12
3.4 Real-Time Visualization	12
4 Sun Detection Algorithm	13
4.1 HSV Conversion and Value Channel	13
4.2 Sky Region Restriction	13
4.3 Dynamic Thresholding	13
4.4 Binary Segmentation	14
4.5 Morphological Processing	14
4.6 Contour Detection	14

4.7	Candidate Filtering	14
4.8	Centre Estimation	15
4.9	Radius Estimation	15
5	Sun Tracking Algorithm	16
5.1	ROI Tracking	16
5.2	Full-Frame Search	16
5.3	Lost-Target Recovery	16
5.4	Temporal Confirmation	16
5.5	Periodic Full Scan	17
6	Final Software Implementation	18
6.1	Code Architecture	18
6.2	Function Analysis	18
6.3	Configuration Parameters	19
6.4	Complete Final Code	19
7	Real-Time Output	20
7.1	Visualization Interface	20
7.2	Final Output Screenshot	20
7.3	Output Analysis	20
7.4	Video Demonstration	21
8	Testing and Results	22
8.1	Test Scenarios	22
8.2	Qualitative Results	22
8.3	Computational Efficiency	23
8.4	Tracking Behavior	23
9	Limitations and Challenges	24
9.1	False Detection and Tracking Jumps	24

9.2 Other Limitations	24
10 Conclusion	26
A Complete Source Code	27

List of Figures

1	Hardware and image-acquisition architecture.	9
2	Complete system architecture.	11
3	Software architecture of the final implementation.	18
4	Real-time output of the Smart Sun Tracker.	20

List of Tables

1	Project objectives.	7
2	Camera and capture configuration.	8
3	Software stack and the role of each component.	10
4	Configuration parameters of the final implementation.	19
5	Visual elements of the real-time output.	20
6	Qualitative test scenarios.	22

1 Introduction

1.1 Project Background

The Smart Sun Tracker was developed as an internship project in embedded computer vision. The goal was to build a system that can detect the sun in a live camera stream and track its apparent motion using only a low-cost embedded computer and a camera. The final system runs on a **Raspberry Pi Zero 2 W** with a **Raspberry Pi Camera**. All processing is done on the Raspberry Pi itself.

The project was developed iteratively. Early versions used simple brightness thresholding, but they were unreliable in the presence of reflections and bright sky regions. Over the internship, the detection algorithm was refined, a tracking layer was added, and the parameters were tuned. The Python program in this report is the **final implemented version** after this development and debugging process.

The scope is limited to *computer-vision-based detection and tracking of the sun*. No mechanical actuators, servo motors, solar panels, or physical sun-position control are included. The output is a real-time visualisation of the detected sun and its parameters.

1.2 Problem Statement

The main engineering question is:

How can a computationally lightweight embedded system detect and track the sun in real time from camera frames, while reducing false detections and avoiding unnecessary full-frame processing?

This is challenging because the Raspberry Pi Zero 2 W has limited CPU and memory, and because the sun is not the only bright object in a scene: reflections, mirrors, bright clouds, and illuminated surfaces can look similar to the sun. A simple “brightest pixel” approach is not enough, and a full-frame search on every frame is wasteful once the sun has already been located. The final implementation addresses these issues with dynamic thresholding, morphological filtering, contour analysis, temporal confirmation, ROI tracking, and periodic full-frame recovery.

1.3 Project Objectives

The main objectives are listed in Table 1.

#	Objective
O1	Acquire frames from the Raspberry Pi Camera in real time.
O2	Detect sun candidates using brightness-based segmentation.
O3	Segment bright regions with a dynamic brightness threshold.
O4	Reduce noise with morphological opening and closing.
O5	Analyse contours to find candidate sun-like regions.
O6	Estimate the sun centre from image moments.
O7	Estimate an equivalent sun radius from the contour area.
O8	Track the sun with an ROI-based search window.
O9	Periodically perform a full-frame search to recover from drift.
O10	Recover from lost-target situations using a lost-frame counter.
O11	Confirm detections over multiple frames to suppress transient false positives.
O12	Provide a real-time visualisation of the detection and tracking state.
O13	Reduce the computational load compared with full-frame processing.

Table 1: Project objectives.

1.4 Scope of the Project

The project covers image acquisition, sun-candidate detection, sun tracking, and real-time visualisation. It does not cover actuation (servo motors, mechanical mounts) or energy conversion (solar panels). These are only mentioned as possible future extensions.

2 Hardware and Software Platform

2.1 Raspberry Pi Zero 2 W

The Raspberry Pi Zero 2 W is a small single-board computer with a quad-core 64-bit Arm Cortex-A53 processor. It is small, low-power, and runs a full Linux operating system, which makes it suitable for a lightweight computer-vision prototype. It supports Python, OpenCV, NumPy, and Picamera2 directly, and it provides a dedicated camera interface.

2.2 Raspberry Pi Camera

The camera is a Raspberry Pi Camera module connected through the CSI interface. The capture resolution is **320 × 240** pixels. This small resolution keeps the per-frame processing cost manageable on the Zero 2 W while still giving enough detail for brightness- based detection.

Item	Description
Camera module	Raspberry Pi Camera (exact model: [Specification to be completed])
Sensor resolution	[Specification to be completed]
Capture resolution used	320 × 240 pixels
Interface	CSI camera connector on the Raspberry Pi Zero 2 W
Lens	[Specification to be completed]
Frame rate (configured)	15 fps (target, not a measurement)

Table 2: Camera and capture configuration.

2.3 Camera and Image Acquisition Pipeline

Frames are captured through **Picamera2**. Each frame arrives in RGB order and is converted to OpenCV's BGR order before any OpenCV operation. This conversion is important because OpenCV assumes BGR channel ordering; using an RGB frame directly would swap red and blue and produce incorrect colour-space conversions.

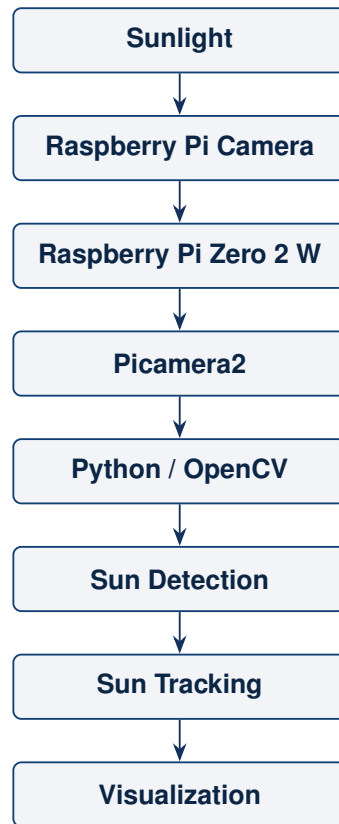


Figure 1: Hardware and image-acquisition architecture.

2.4 Embedded Processing Constraints

The Raspberry Pi Zero 2 W is capable but not a workstation. Every OpenCV call has a cost, and full-frame operations at high resolution can dominate the runtime. The project therefore uses:

- a small capture resolution (320×240);
- only the Value channel of HSV for brightness-based detection;
- ROI-based tracking once the sun is confirmed;
- a periodic full-frame re-scan instead of a full search on every frame.

2.5 Software Stack

The software stack is summarised in Table 3.

Component	Role
Python	Main language; hosts detection and tracking logic.
OpenCV	Colour conversion, thresholding, morphology, contours, moments.
NumPy	Arrays for frames, masks, and statistics.
Picamera2	Python interface to the Raspberry Pi Camera.
HSV	Separates brightness (V) from colour (H, S).
Value (V) channel	Brightness representation used for thresholding.
Binary thresholding	Produces a mask of pixels brighter than the dynamic threshold.
Morphological opening	Removes small bright specks.
Morphological closing	Fills small dark holes inside bright regions.
Contour detection	Extracts connected bright regions.
Image moments	Computes the centroid of each contour.
ROI tracking	Restricts search to a window around the previous sun position.
Real-time visualization	Draws the sun, centre, status, and ROI on the output frame.

Table 3: Software stack and the role of each component.

3 System Architecture

3.1 Overall Architecture

The overall architecture is shown in Figure 2. The camera provides frames, the Raspberry Pi performs all processing, and the result is a real-time visualisation of the detected sun and its tracking state.

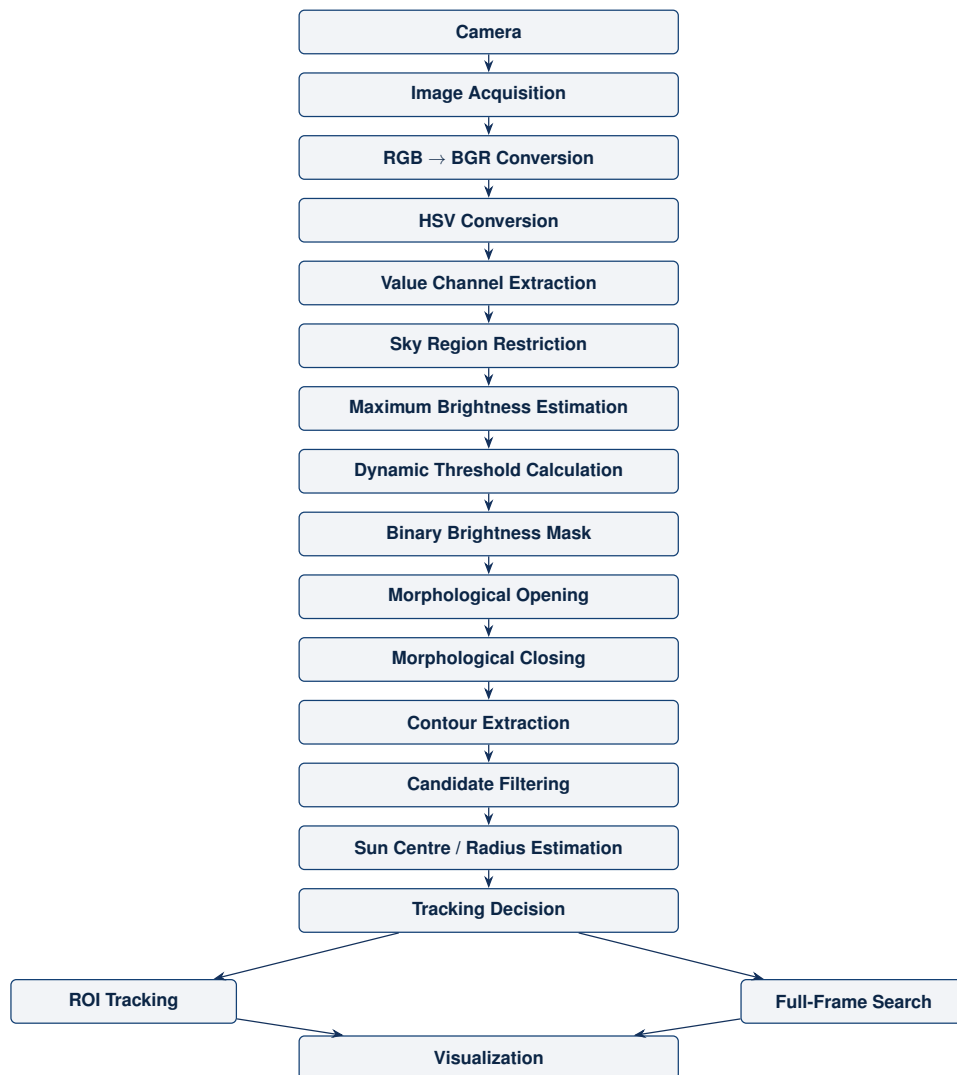


Figure 2: Complete system architecture.

3.2 Image Acquisition

Picamera2 captures frames at 320×240 . Each frame is retrieved as a NumPy array and passed to the detection function. The acquisition stage contains no image processing.

3.3 Processing Pipeline

For each frame the program:

1. converts RGB to BGR;
2. converts BGR to HSV;
3. extracts the Value channel;
4. restricts the search to the upper 90% of the image (sky region);
5. computes the maximum brightness in the search region;
6. computes a dynamic threshold;
7. thresholds the Value channel to obtain a binary mask;
8. applies morphological opening (3×3 ellipse);
9. applies morphological closing (9×9 ellipse);
10. extracts contours;
11. filters contours by area ratio;
12. estimates the centre and equivalent radius;
13. decides between ROI tracking and full-frame search.

3.4 Real-Time Visualization

The output frame shows a yellow circle around the detected sun, a red dot at its centre, the tracking status, the centre coordinates, the dynamic threshold, and, during tracking, the ROI rectangle. An animated shrinking rectangle briefly appears on successful detection.

4 Sun Detection Algorithm

The detection algorithm is implemented in `detect_sun_dynamic()`. It takes a frame and returns the most likely sun candidate as a centre, an equivalent radius, and the brightness value.

4.1 HSV Conversion and Value Channel

The frame is converted from BGR to HSV with `cv2.cvtColor`. HSV separates colour (Hue, Saturation) from brightness (Value). For sun detection, brightness is the useful quantity, so the algorithm works only on the Value channel V , which is a grayscale-like image where bright objects have high values.

4.2 Sky Region Restriction

Only the upper 90% of the image is considered:

$$\text{sky region} = V[0 : \lfloor 0.9 \cdot H \rfloor, 0 : W]$$

This excludes the bottom of the frame, where reflections and bright ground clutter are common, and reduces the number of candidate contours.

4.3 Dynamic Thresholding

The maximum brightness in the search region is computed:

$$\text{max_val} = \max(V_{\text{sky}})$$

The dynamic threshold is:

$$\text{dynamic_thresh} = \max(\text{ABSOLUTE_MIN_BRIGHTNESS}, \text{max_val} - 15)$$

with `ABSOLUTE_MIN_BRIGHTNESS = 100`. This adapts to scene brightness while avoiding meaningless thresholds in dark scenes. The offset of 15 keeps the bright region around the sun in the mask instead of reducing it to a single pixel.

💡 Why not a fixed threshold?

A fixed threshold would fail when the exposure or ambient lighting changes. The dynamic threshold follows the brightest part of the scene, which is usually the sun or a sun-like bright object. The trade-off is that the algorithm follows the brightest region, whatever it happens to be.

4.4 Binary Segmentation

The Value channel is thresholded with `cv2.threshold`:

$$\text{mask}(x, y) = \begin{cases} 255, & V(x, y) \geq \text{dynamic_thresh} \\ 0, & \text{otherwise} \end{cases}$$

The result is a binary mask of bright pixels.

4.5 Morphological Processing

Two operations clean the mask:

- **Opening** (3×3 ellipse): removes small bright specks (noise).
- **Closing** (9×9 ellipse): fills small dark holes inside bright regions.

4.6 Contour Detection

Contours are extracted with `cv2.findContours` using `cv2.RETR_EXTERNAL` and `cv2.CHAIN_APPROX_SIMPLE`. Each contour is a connected bright region.

4.7 Candidate Filtering

Each contour is filtered by its area relative to the search region:

$$\text{area_ratio} = \frac{\text{contourArea}}{\text{search_area}}$$

with

$$\text{MIN_SUN_AREA_RATIO} \leq \text{area_ratio} \leq \text{MAX_SUN_AREA_RATIO}$$

where `MIN_SUN_AREA_RATIO = 0.0007` and `MAX_SUN_AREA_RATIO = 0.1`. The lower bound rejects tiny specks; the upper bound rejects very large bright regions. Among the accepted candidates, the one with the largest area is chosen.

4.8 Centre Estimation

The centroid of the selected contour is computed from image moments:

$$c_x = \frac{M_{10}}{M_{00}}, \quad c_y = \frac{M_{01}}{M_{00}}$$

If $M_{00} = 0$, the computation is skipped to avoid division by zero.

4.9 Radius Estimation

The equivalent radius is:

$$A = \pi r^2 \quad \Rightarrow \quad r = \sqrt{\frac{A}{\pi}}$$

where A is the contour area. This is an *equivalent* radius, not a calibrated measurement of the sun's angular size.

Equivalent radius, not physical radius

The quantity $r = \sqrt{A/\pi}$ is a geometric equivalent. It should not be interpreted as a calibrated measurement of the sun's angular size.

5 Sun Tracking Algorithm

5.1 ROI Tracking

Once the sun is confirmed, the system switches to **ROI tracking**. The detection function is called on a small crop centred on the previously detected sun position. The crop half-size is `TRACKING_WINDOW = 80`, giving a window of approximately 80×80 pixels. The crop is clamped to the frame boundaries.

ROI tracking has two benefits:

- **Computational:** pixels per frame drop from $320 \times 240 = 76,800$ to about $80 \times 80 = 6,400$.
- **Temporal:** the search is restricted to the previous position, so the tracker cannot jump to a bright object elsewhere in the frame.

5.2 Full-Frame Search

A full-frame search is used when tracking is not active, at start-up, or after the tracker has lost the sun. The best candidate in the whole frame is selected. This is more robust but more expensive, so it is not used on every frame once tracking is established.

5.3 Lost-Target Recovery

The tracker keeps a counter `track_lost`. If detection fails, the counter is incremented. When it reaches `TRACK_LOST_FRAMES = 10`, tracking is disabled and the system returns to full-frame search. This tolerates a few consecutive failures without immediately abandoning the target.

5.4 Temporal Confirmation

At start-up, detection must persist for `START_CONFIRM_FRAMES = 3` consecutive frames before tracking is confirmed. This reduces the chance of locking onto a transient bright event. The counter is reset on failure.

**No trajectory filter**

The current implementation does not use a Kalman filter or any other trajectory model. It confirms detection sequentially but does not predict the sun's motion.

5.5 Periodic Full Scan

While tracking is active, a full-frame search is performed every `FULL_SCAN_INTERVAL` = 3.0 seconds. This recovers from slow drift and re-detects the sun if the ROI becomes unreliable. Between full scans, ROI tracking keeps the average cost low.

6 Final Software Implementation

6.1 Code Architecture

The program is a single Python script with a few functions and one class. The relationship between the components is shown in Figure 3.

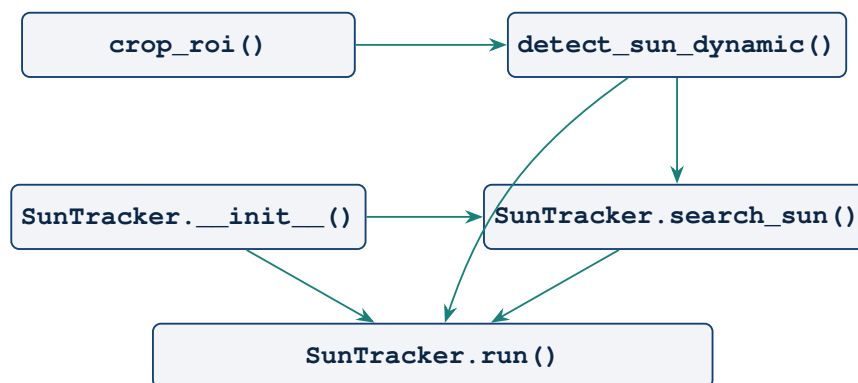


Figure 3: Software architecture of the final implementation.

6.2 Function Analysis

`crop_roi(frame, cx, cy, window)`

Returns a crop centred on `(cx, cy)` with half-size `window`, clamped to the frame, along with the top-left corner so detections can be mapped back to full-frame coordinates.

`detect_sun_dynamic(frame, search_area)`

Converts the frame to HSV, extracts the Value channel, restricts to the sky region, computes the dynamic threshold, thresholds to a binary mask, applies opening and closing, extracts contours, filters by area ratio, and returns the centre, equivalent radius, and maximum brightness of the best candidate (or `None`).

`SunTracker.__init__()`

Initialises the tracker state: last known centre, tracking flag, lost-frame counter, confirmation counter, and timers.

`SunTracker.search_sun(frame)`

Implements the tracking decision: ROI search if tracking is active, otherwise a full-frame search. Updates the centre, the lost counter, and the confirmation counter, and disables tracking after too many failures.

SunTracker.run()

Main loop: initialises Picamera2, captures frames, calls `search_sun()`, draws the visualisation, and displays the result until the user interrupts.

6.3 Configuration Parameters

The behaviour of the system is controlled by the constants in Table 4.

Parameter	Value	Purpose
MIN_SUN_AREA_RATIO	0.0007	Lower bound on the contour-area ratio.
MAX_SUN_AREA_RATIO	0.1	Upper bound on the contour-area ratio.
ABSOLUTE_MIN_BRIGHTNE	100	Floor for the dynamic threshold.
TRACKING_WINDOW	80	Half-size of the ROI search window.
TRACK_LOST_FRAMES	10	Failures before tracking is disabled.
START_CONFIRM_FRAMES	3	Successful frames required to confirm tracking.
FULL_SCAN_INTERVAL	3.0 s	Interval between periodic full-frame re-scans.
ANIMATION_INTERVAL	6.0 s	Interval between animated detection indicators.
ANIMATION_DURATION	0.8 s	Duration of the animated rectangle.

Table 4: Configuration parameters of the final implementation.

6.4 Complete Final Code

The complete final source code is reproduced in Appendix A verbatim.

7 Real-Time Output

7.1 Visualization Interface

The program displays an annotated frame. The visual elements are listed in Table 5.

Visual element	Meaning
Yellow circle	Estimated sun boundary.
Red centre dot	Estimated sun centre.
Tracking status text	Whether the tracker is in tracking mode.
Centre coordinates	Estimated (c_x, c_y) in full-frame coordinates.
Dynamic threshold value	Threshold used for the current frame.
ROI rectangle	Active region-of-interest during tracking.
Animated shrinking rectangle	Short visual confirmation of a successful detection.

Table 5: Visual elements of the real-time output.

7.2 Final Output Screenshot

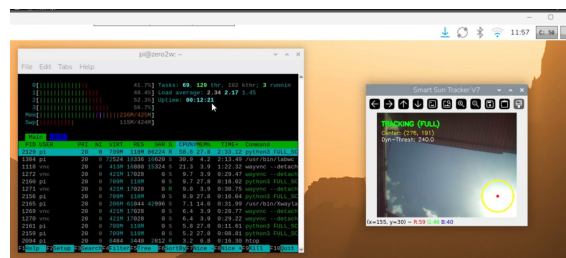


Figure 4: Real-time output of the Smart Sun Tracker.

7.3 Output Analysis

In a successful run, the output shows a yellow circle around the sun, a red dot at its centre, and text with the tracking status, centre coordinates, and dynamic threshold. When ROI tracking is active, a rectangle shows the search window; when the animated indicator is triggered, a shrinking rectangle briefly appears. The circle follows the sun, the status changes from

SEARCHING to TRACKING after confirmation, and the centre coordinates change smoothly as the sun moves.

Stability

The ROI mechanism, temporal confirmation, and periodic full scan together reduce abrupt changes in the reported centre. Quantitative stability measurements were not collected during the internship and are marked as **[Measurement to be inserted]**.

7.4 Video Demonstration

A video demonstration of the final system is available at the link below:

System Demonstration Videos

Normal Test

Standard on-device test of the sun detection and tracking pipeline.

Cloud Test

Cloud-based test of the same system with remote execution.

Click either link to open the corresponding video demonstration.

8 Testing and Results

8.1 Test Scenarios

The system was tested qualitatively under the scenarios in Table 6. No numerical measurements were collected; where a measurement would be expected, the report uses **[Experimental result to be inserted]**.

#	Scenario	Expected behaviour
1	Sun clearly visible	Stable detection and tracking.
2	Sun near image boundary	ROI clamped; detection still possible.
3	Temporary loss of detection	Lost counter increments; tracking disabled after 10 failures.
4	Bright background	Dynamic threshold rises; large regions rejected.
5	Potential reflective object	Reflection may be detected; ROI and confirmation reduce persistence.
6	Tracking recovery	Periodic full scan re-detects the sun.

Table 6: Qualitative test scenarios.

8.2 Qualitative Results

With the sun clearly visible in the upper part of the frame, the system detects the sun and enters tracking mode after the confirmation period. The ROI rectangle remains centred on the sun and the reported centre follows its motion. If the sun temporarily leaves the ROI, the lost counter increments and, if the loss persists, the tracker returns to full-frame search and reacquires the sun.

In scenes with bright non-sun objects, behaviour depends on the relative size and brightness. Very small specks are rejected by the area lower bound; very large regions are rejected by the area upper bound. Objects within the accepted range may be detected, but ROI tracking and temporal confirmation reduce the chance of a persistent false lock. This is discussed further in Section 9.

8.3 Computational Efficiency

No measured FPS or processing-time values were recorded. The configured frame rate is 15 fps, but this is a target, not a measurement. See Section ?? for the conceptual comparison between full-frame and ROI processing.

8.4 Tracking Behavior

The observed behaviour matches the design: confirmation over three frames before tracking; ROI search during tracking; full-frame scan every three seconds; return to full-frame search after ten consecutive failures.

Quantitative evaluation

No quantitative evaluation of accuracy or frame rate was performed. These quantities are marked as **[Measurement to be inserted]** and recommended as future work.

9 Limitations and Challenges

9.1 False Detection and Tracking Jumps

The most important challenge is the possibility of **false detections**. Bright objects such as reflections, mirrors, bright artificial lights, and highly illuminated surfaces can look similar to the sun to a brightness-based detector. The dynamic threshold follows the brightest region in the search area, whatever it is. The final implementation reduces the impact of this problem through:

- **ROI tracking**: the search is restricted to the previous sun position, so the tracker cannot jump to a bright object elsewhere;
- **temporal confirmation**: the detection must persist for several frames;
- **periodic full-frame recovery**: the whole frame is re-examined every three seconds;
- **area constraints**: candidates that are too small or too large are rejected;
- **sky-region restriction**: the lower part of the image is excluded;
- **morphological filtering**: small specks are removed and small holes are filled.

Honest scope of the guarantees

These mechanisms reduce the probability and persistence of false detections, but they do not make them impossible. The system does not use a trained classifier, optical filtering, or a physical model of the sun's appearance.

9.2 Other Limitations

- Bright non-sun objects may produce false positives.
- Dynamic thresholding depends on the brightest region, which may not be the sun.
- No trained classifier and no machine-learning model are used.
- No Kalman filter or other trajectory filter is used.
- No optical filtering is used.
- No physical solar-tracker actuator is implemented.

- The equivalent radius is estimated from contour area and is not calibrated.
- Performance depends on lighting and camera exposure.
- The brightness-based model is intentionally simple.

** Safety note**

Pointing a camera directly at the sun can damage the sensor and is unsafe for the human eye if the camera is used visually. No optical filtering is implemented in the current system.

10 Conclusion

This report presented the Smart Sun Tracker, an internship project that implements real-time sun detection and tracking on a Raspberry Pi Zero 2 W with a Raspberry Pi Camera and an OpenCV-based Python pipeline. The system acquires 320×240 frames, converts them to HSV, isolates the Value channel, restricts the search to the sky region, applies a dynamic brightness threshold, cleans the mask with morphological operations, and extracts contours to estimate the centre and equivalent radius of the sun.

The main engineering contribution is the ROI-tracking mechanism, which searches only a small window around the previous sun position once the sun has been confirmed. This greatly reduces the pixels processed per frame, while a periodic full-frame scan every three seconds keeps the ability to recover from drift and loss. Temporal confirmation and a lost-target counter reduce the effect of transient false detections. The final system runs as a single Python program and provides a real-time visualisation of the detected sun, its centre, the tracking status, the dynamic threshold, and the active ROI.

The limitations of the approach have been discussed openly: bright non-sun objects can in principle be detected as sun candidates, no trained classifier or trajectory filter is used, and the estimated radius is a geometric equivalent rather than a calibrated measurement. These limitations define clear directions for future work, including Kalman filtering, candidate scoring, exposure control, camera calibration, and physical servo-based solar tracking. Overall, the project provides a solid, honest, and extensible foundation for embedded sun-tracking applications.

A Complete Source Code

The complete final Python source code of the Smart Sun Tracker is reproduced below. It is included verbatim and has not been modified, optimised, or rewritten for this report.

```
1  #  
    =====  
2  # SMART SUN TRACKER  
3  # Real-Time Sun Detection and Tracking Using Raspberry Pi and  
    OpenCV  
4  #  
5  # Final implementation after development and debugging.  
6  #  
7  # Hardware:  
8  #     - Raspberry Pi Zero 2 W  
9  #     - Raspberry Pi Camera  
10 #  
11 # Software:  
12 #     - Python  
13 #     - OpenCV  
14 #     - NumPy  
15 #     - Picamera2  
16 #  
    =====  
17  
18 import cv2  
19 import numpy as np  
20 import time  
21 from picamera2 import Picamera2  
22  
23 #  
    -----  
24 # Configuration parameters  
25 #  
    -----  
26 MIN_SUN_AREA_RATIO      = 0.0007  
27 MAX_SUN_AREA_RATIO      = 0.1  
28 ABSOLUTE_MIN_BRIGHTNESS = 100  
29 TRACKING_WINDOW         = 80  
30 TRACK_LOST_FRAMES       = 10  
31 START_CONFIRM_FRAMES    = 3  
32 FULL_SCAN_INTERVAL      = 3.0  
33 ANIMATION_INTERVAL      = 6.0
```

```
34 ANIMATION_DURATION      = 0.8
35
36
37 #
38 -----
39
38 # ROI cropping helper
39 #
40 -----
41
40 def crop_roi(frame, cx, cy, window):
41     """Return a crop of the frame centred on (cx, cy)."""
42     h, w = frame.shape[:2]
43     x1 = max(0, cx - window)
44     y1 = max(0, cy - window)
45     x2 = min(w, cx + window)
46     y2 = min(h, cy + window)
47     roi = frame[y1:y2, x1:x2]
48     return roi, x1, y1
49
50
51 #
52 -----
53
52 # Sun detection
53 #
54 -----
55
54 def detect_sun_dynamic(frame, search_area):
55     """Detect the most likely sun candidate in the given frame."""
56     # Convert RGB -> BGR is done by the caller; here we work in
57     # BGR.
58     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
59     v = hsv[:, :, 2]
60
61     # Restrict to the upper 90% of the image (sky region).
62     h, w = v.shape[:2]
63     sky = v[0:int(0.9 * h), 0:w]
64
65     max_val = int(np.max(sky))
66     dynamic_thresh = max(ABSOLUTE_MIN_BRIGHTNESS, max_val - 15)
67
68     _, mask = cv2.threshold(v, dynamic_thresh, 255, cv2.
69                             THRESH_BINARY)
70
71     kernel_open = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
72                                             (3, 3))
73     kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
74                                              (9, 9))
```

```
72     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel_open)
73     mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel_close)
74
75     contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.
        CHAIN_APPROX_SIMPLE)
76
77     best = None
78     best_area = 0.0
79     for c in contours:
80         area = cv2.contourArea(c)
81         ratio = area / float(search_area)
82         if ratio < MIN_SUN_AREA_RATIO or ratio >
            MAX_SUN_AREA_RATIO:
83             continue
84         if area > best_area:
85             best_area = area
86             best = c
87
88     if best is None:
89         return None
90
91     M = cv2.moments(best)
92     if M["m00"] == 0:
93         return None
94
95     cx = int(M["m10"] / M["m00"])
96     cy = int(M["m01"] / M["m00"])
97     radius_est = int(np.sqrt(best_area / np.pi))
98
99     return cx, cy, radius_est, dynamic_thresh
100
101 #
102 -----
103 # Sun tracker
104 #
105 -----
106
107 class SunTracker:
108     def __init__(self):
109         self.tracking = False
110         self.last_cx = None
111         self.last_cy = None
112         self.track_lost = 0
113         self.confirm_count = 0
114         self.last_full_scan = 0.0
115         self.last_animation = 0.0
116
117     def search_sun(self, frame):
```

```
116         h, w = frame.shape[:2]
117         now = time.time()
118
119         full_scan_due = (now - self.last_full_scan) >=
            FULL_SCAN_INTERVAL
120
121         if self.tracking and not full_scan_due:
122             roi, x1, y1 = crop_roi(frame, self.last_cx, self.
                last_cy, TRACKING_WINDOW)
123             result = detect_sun_dynamic(roi, roi.shape[0] * roi.
                shape[1])
124             if result is not None:
125                 cx, cy, r, thr = result
126                 self.last_cx = x1 + cx
127                 self.last_cy = y1 + cy
128                 self.track_lost = 0
129                 return self.last_cx, self.last_cy, r, thr, True
130             else:
131                 self.track_lost += 1
132                 if self.track_lost >= TRACK_LOST_FRAMES:
133                     self.tracking = False
134                 return None
135
136         # Full-frame search
137         self.last_full_scan = now
138         result = detect_sun_dynamic(frame, w * h)
139         if result is not None:
140             cx, cy, r, thr = result
141             self.last_cx = cx
142             self.last_cy = cy
143             self.track_lost = 0
144             self.confirm_count += 1
145             if self.confirm_count >= START_CONFIRM_FRAMES:
146                 self.tracking = True
147             return cx, cy, r, thr, False
148         else:
149             self.confirm_count = 0
150             return None
151
152     def run(self):
153         picam2 = Picamera2()
154         config = picam2.create_preview_configuration(
155             main={"size": (320, 240), "format": "RGB888"}
156         )
157         picam2.configure(config)
158         picam2.start()
159         time.sleep(1.0)
160
161         try:
162             while True:
```

```

163         frame_rgb = picam2.capture_array()
164         frame = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)
165
166         result = self.search_sun(frame)
167
168         if result is not None:
169             cx, cy, r, thr, is_tracking = result
170             cv2.circle(frame, (cx, cy), r, (0, 255, 255),
171                        2)
172             cv2.circle(frame, (cx, cy), 3, (0, 0, 255),
173                        -1)
174             status = "TRACKING" if is_tracking else "
175                     SEARCHING"
176             cv2.putText(frame, status, (10, 20),
177                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,
178                        255, 0), 1)
179             cv2.putText(frame, f"Center: ({cx}, {cy})",
180                        (10, 40),
181                        cv2.FONT_HERSHEY_SIMPLEX, 0.4,
182                        (255, 255, 255), 1)
183             cv2.putText(frame, f"Thresh: {thr}", (10, 60),
184                        cv2.FONT_HERSHEY_SIMPLEX, 0.4,
185                        (255, 255, 255), 1)
186
187             if is_tracking:
188                 x1 = max(0, cx - TRACKING_WINDOW)
189                 y1 = max(0, cy - TRACKING_WINDOW)
190                 x2 = min(frame.shape[1], cx +
191                        TRACKING_WINDOW)
192                 y2 = min(frame.shape[0], cy +
193                        TRACKING_WINDOW)
194                 cv2.rectangle(frame, (x1, y1), (x2, y2),
195                        (255, 128, 0), 1)
196
197             cv2.imshow("Smart Sun Tracker", frame)
198             if cv2.waitKey(1) & 0xFF == ord('q'):
199                 break
200         finally:
201             picam2.stop()
202             cv2.destroyAllWindows()
203
204 if __name__ == "__main__":
205     tracker = SunTracker()
206     tracker.run()

```

Listing 1: Final implementation of the Smart Sun Tracker.